

Module – 1

BASIC STRUCTURE OF COMPUTERS

BASIC CONCEPTS

- Computer Architecture (CA) is concerned with the structure and behavior of the computer.
 - CA includes the information formats, the instruction set and techniques for addressing memory.
 - In general covers, CA covers 3 aspects of computer-design namely: 1) Computer Hardware, 2) Instruction set Architecture and 3) Computer Organization.
- 1. **Computer Hardware** - It consists of electronic circuits, displays, and magnetic and optical storage media and communication facilities.
- 2. **Instruction Set Architecture** - It is programmer visible machine interface such as instruction set, registers, memory organization and exception handling.

Two main approaches are 1) CISC and 2) RISC.

(CISC- Complex Instruction Set Computer, RISC-Reduced Instruction Set Computer)

3. Computer Organization

It includes the high level aspects of a design, such as

- Memory-system
- Bus-structure &
- Design of the internal CPU.

FUNCTIONAL UNITS

A computer consists of 5 functionally independent main parts:

- 1) Input 2) Memory 3) ALU 4) Output 5) Control units.

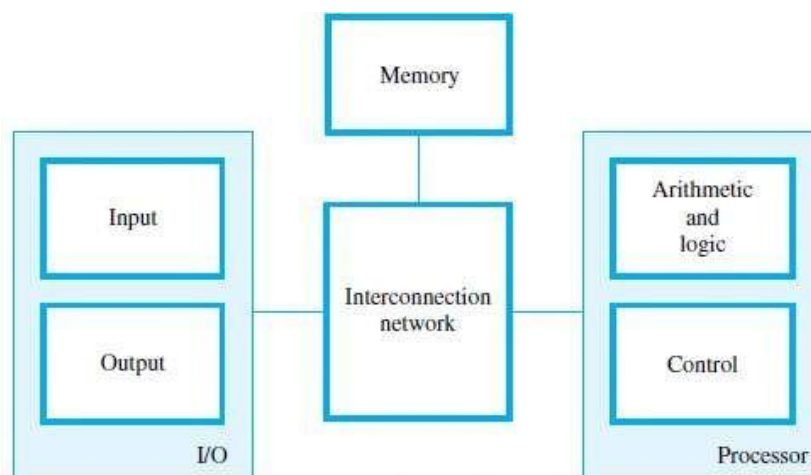


Figure 1.1 Basic functional units of a computer.

BASIC OPERATIONAL CONCEPTS

An Instruction consists of 2 parts, 1) Operation code (Opcode) and 2) Operands.

OPCODE	OPERANDS
--------	----------

- The data/operands are stored in memory.
- The individual instructions are brought from the memory to the processor.
- Then, the processor performs the specified operation.

Let us see a typical instruction

ADD LOCA, R0

This instruction is an addition operation. The following are the steps to execute the instruction:

- Step 1: Fetch the instruction from main-memory into the processor.
- Step 2: Fetch the operand at location LOCA from main-memory into the processor.
- Step 3: Add the memory operand (fetched contents of LOCA) to the contents of register R0.
- Step 4: Store the result (sum) in R0.

The same instruction can be realized using 2 instructions as:

Load LOCA, R1

Add R1, R0

The following are the steps to execute the instruction:

- Step 1: Fetch the instruction from main-memory into the processor.
- Step 2: Fetch the operand at location LOCA from main-memory into the register R1.
- Step 3: Add the content of Register R1 and the contents of register R0.
- Step 4: Store the result (sum) in R0.

MAIN PARTS OF PROCESSOR

- The processor contains ALU, control-circuitry and many registers.
- The processor contains “n” general-purpose registers R0 through Rn-1.
- The IR holds the instruction that is currently being executed.
- The control-unit generates the timing-signals that determine when a given action is to take place.
- The PC contains the memory-address of the next-instruction to be fetched & executed.
- During the execution of an instruction, the contents of PC are updated to point to next instruction.
- The MAR holds the address of the memory-location to be accessed.
- The MDR contains the data to be written into or read out of the addressed location.
- MAR and MDR facilitates the communication with memory.

(IR- Instruction-Register, PC- Program Counter, MAR-Memory Address Register, MDR-Memory Data Register)

STEPS TO EXECUTE AN INSTRUCTION

1. The address of first instruction (to be executed) gets loaded into PC.
2. The contents of PC (i.e. address) are transferred to the MAR & control-unit issues Read signal to memory.
3. After certain amount of elapsed time, the first instruction is read out of memory and placed into MDR.
4. Next, the contents of MDR are transferred to IR. At this point, the instruction can be decoded & executed.
5. To fetch an operand, it's address is placed into MAR & control-unit issues Read signal. As a result, the operand is transferred from memory into MDR, and then it is transferred from MDR to ALU.
6. Likewise required number of operands is fetched into processor.
7. Finally, ALU performs the desired operation.
8. If the result of this operation is to be stored in the memory, then the result is sent to the MDR.
9. The address of the location where the result is to be stored is sent to the MAR and a Write cycle is initiated.
10. At some point during execution, contents of PC are incremented to point to next instruction in the program.

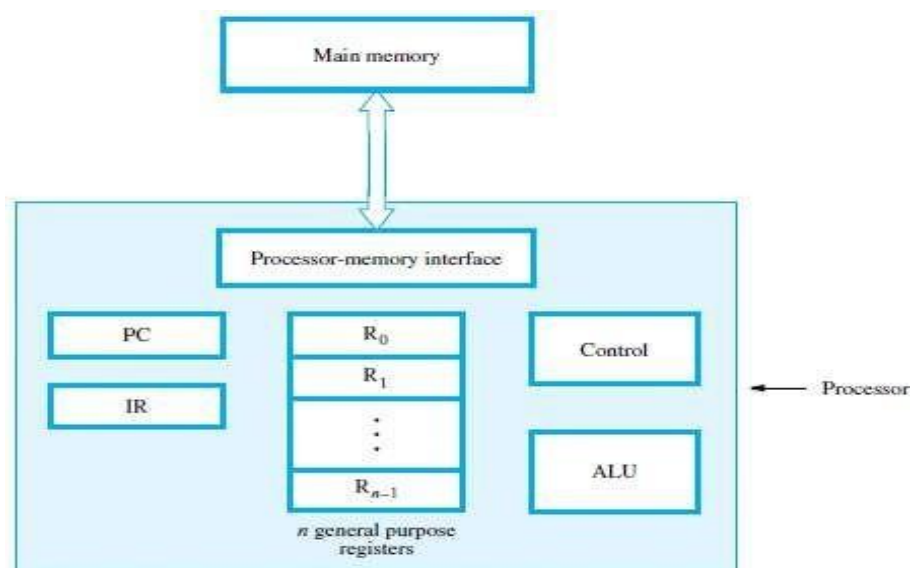


Figure 1.2 Connection between the processor and the main memory.

BUS STRUCTURE

A bus is a group of lines that serves as a connecting path for several devices. A bus may be lines or wires. The lines carry data or address or control signal.

There are 2 types of Bus structures: 1) Single Bus Structure and 2) Multiple Bus Structure.

1) Single Bus Structure

➤ Because the bus can be used for only one transfer at a time, only 2 units can actively use the bus at any given time.

➤ Bus control lines are used to arbitrate multiple requests for use of the bus.

➤ **Advantages:**

Low cost & Flexibility for attaching peripheral devices.

2) Multiple Bus Structure

- Systems that contain multiple buses achieve more concurrency in operations.
- Two or more transfers can be carried out at the same time.
- **Advantage:** Better performance.
- **Disadvantage:** Increased cost.

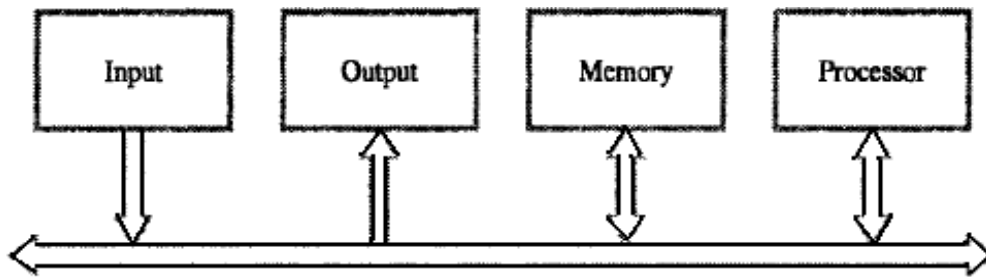


Figure 1.3 Single-bus structure.

- The devices connected to a bus vary widely in their speed of operation.
- To synchronize their operational-speed, buffer-registers can be used.
- **Buffer Registers**
 - are included with the devices to hold the information during transfers.
 - prevent a high-speed processor from being locked to a slow I/O device during data transfers.

PERFORMANCE

- The most important measure of performance of a computer is how quickly it can execute programs.
- The speed of a computer is affected by the design of
 - 1) Instruction-set.
 - 2) Hardware & the technology in which the hardware is implemented.
 - 3) Software including the operating system.
- Because programs are usually written in a HLL, performance is also affected by the compiler that translates programs into machine language. (HLL → High Level Language).
- For best performance, it is necessary to design the compiler, machine instruction set and hardware in a co-ordinated way.

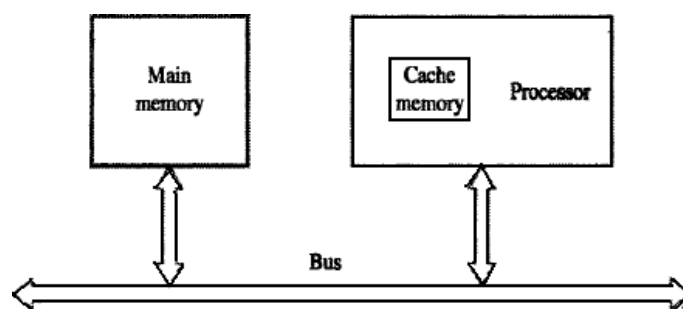


Figure 1.5 The processor cache.

Let examine the flow of program instructions and data between the memory & the processor.

- At the start of execution, all program instructions are stored in the main-memory.
- As execution proceeds, instructions are fetched into the processor, and a copy is placed in the cache.
- Later, if the same instruction is needed a second time, it is read directly from the cache.
- A program will be executed faster
 - if movement of instruction/data between the main-memory and the processor is minimized which is achieved by using the cache.

PROCESSOR CLOCK

- Processor circuits are controlled by a timing signal called a **Clock**.
- The clock defines regular time intervals called **Clock Cycles**.
- To execute a machine instruction, the processor divides the action to be performed into a sequence of basic steps such that each step can be completed in one clock cycle.
- Let P = Length of one clock cycle
 R = Clock rate.

Relation between P and R is given by $R = \frac{1}{P}$

- R is measured in cycles per second.
- Cycles per second is also called Hertz (Hz)

BASIC PERFORMANCE EQUATION

- Let T = Processor time required to executed a program. N = Actual number of instruction executions.
 S = Average number of basic steps needed to execute one machine instruction. R = Clock rate in cycles per second.
- The program execution time is $T = \frac{N \times S}{R}$ given by
- Equ1 is referred to as the basic performance equation.
- To achieve high performance, the computer designer must reduce the value of T , which means reducing N and S , and increasing R .
 - The value of N is reduced if source program is compiled into fewer machine instructions.
 - The value of S is reduced if instructions have a smaller number of basic steps to perform.
 - The value of R can be increased by using a higher frequency clock.
- Care has to be taken while modifying values since changes in one parameter may affect the other.

CLOCK RATE

- There are 2 possibilities for increasing the clock rate R :
 - 1) Improving the IC technology makes logic-circuits faster.
This reduces the time needed to compute a basic step. (IC → integrated circuits).
This allows the clock period P to be reduced and the clock rate R to be increased.
 - 2) Reducing the amount of processing done in one basic step also reduces the clock period P .
- In presence of a cache, the percentage of accesses to the main-memory is small.
Hence, much of performance-gain expected from the use of faster technology can be realized.
The value of T will be reduced by same factor as R is increased „,“ S & N are not affected.

PERFORMANCE MEASUREMENT

- Benchmark refers to standard task used to measure how well a processor operates.
- The Performance Measure is the time taken by a computer to execute a given benchmark.
- SPEC selects & publishes the standard programs along with their test results for different application domains. (SPEC → System Performance Evaluation Corporation).
- SPEC rating is given by

$$\text{SPEC rating} = \frac{\text{Running time on the reference computer}}{\text{Running time on the computer under test}}$$

SPEC rating = 50 → The computer under test is 50 times as fast as reference-computer.

- The test is repeated for all the programs in the SPEC suite. Then, the geometric mean of the results is computed.

- Let $SPEC_i$ = Rating for program „i’ in the suite.
Overall SPEC rating for the computer is given by

$$SPEC\ rating = \left(\prod_{i=1}^n SPEC_i \right)^{\frac{1}{n}}$$

where n = no. of programs in the suite.

Problem 1:

List the steps needed to execute the machine instruction:

Load R2, LOC

in terms of transfers between the components of processor and some simple control commands.

Assume that the address of the memory-location containing this instruction is initially in register PC.

Solution:

1. Transfer the contents of register PC to register MAR.
2. Issue a Read command to memory.
And, then wait until it has transferred the requested word into register MDR.
3. Transfer the instruction from MDR into IR and decode it.
4. Transfer the address LOCA from IR to MAR.
5. Issue a Read command and wait until MDR is loaded.
6. Transfer contents of MDR to the ALU.
7. Transfer contents of R0 to the ALU.
8. Perform addition of the two operands in the ALU and transfer result into R0.
9. Transfer contents of PC to ALU.
10. Add 1 to operand in ALU and transfer incremented address to PC.

Problem 2:

A program contains 1000 instructions. Out of that 25% instructions requires 4 clock cycles, 40% instructions requires 5 clock cycles and remaining require 3 clock cycles for execution. Find the total time required to execute the program running in a 1 GHz machine.

Solution:

$N = 1000$

25% of $N = 250$ instructions require 4 clock cycles.

40% of $N = 400$ instructions require 5 clock cycles. 35% of $N = 350$ instructions require 3 clock cycles.

$$T = (N \times S) / R = (250 \times 4 + 400 \times 5 + 350 \times 3) / 1 \times 10^9 = (1000 + 2000 + 1050) / 1 \times 10^9 = 4.05 \mu s.$$

Problem 3:

For the following processor, obtain the performance.

Clock rate = 800 MHz

No. of instructions executed = 1000

Average no of steps needed / machine instruction = 20

Solution:

$$T = \frac{N \times S}{R} = (1000 \times 20) / 800 \times 10^6 = 25 \text{ micro sec or } 25 \times 10^{-6} \text{ sec}$$

MEMORY-LOCATIONS & ADDRESSES

- **Memory** consists of many millions of storage cells (flip-flops).
- Each cell can store a bit of information i.e. 0 or 1 (Figure 2.1).
- Each group of n bits is referred to as a **word** of information, and n is called the **word length**.

- The word length can vary from 8 to 64 bits.
- A unit of 8 bits is called a **byte**.
- Accessing the memory to store or retrieve a single item of information (word/byte) requires distinct addresses for each item location. (It is customary to use numbers from 0 through 2^k-1 as the addresses of successive-locations in the memory).
- If 2^k = no. of addressable locations;
then 2^k addresses constitute the address-space of the computer.

For example, a 24-bit address generates an address-space of 2^{24} locations (16 MB).

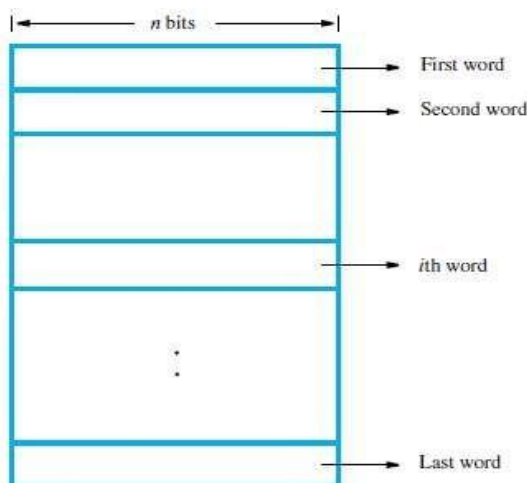


Figure 2.1 Memory words.

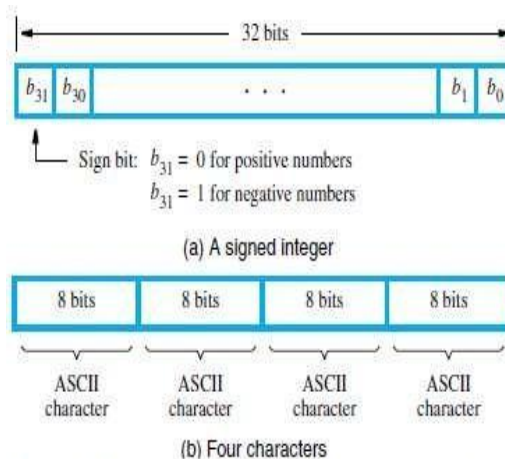


Figure 2.2 Examples of encoded information in a 32-bit word.

BYTE-ADDRESSABILITY

- In byte-addressable memory, successive addresses refer to successive byte locations in the memory.
- Byte locations have addresses 0, 1, 2,
- If the word-length is 32 bits, successive words are located at addresses 0, 4, 8, . . with each word having 4 bytes.

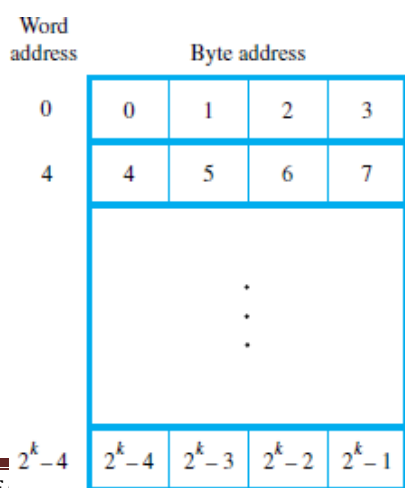
BIG-ENDIAN & LITTLE-ENDIAN ASSIGNMENTS

- There are two ways in which byte-addresses are arranged (Figure 2.3).

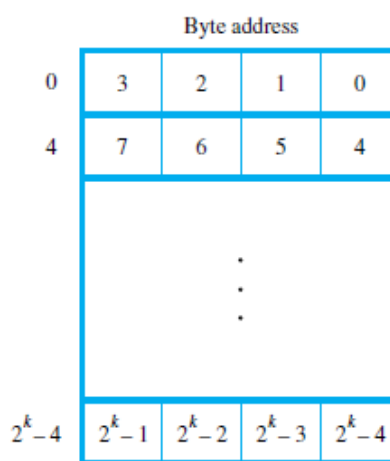
1) Big-Endian: Lower byte-addresses are used for the more significant bytes of the word.

2) Little-Endian: Lower byte-addresses are used for the less significant bytes of the word

- In both cases, byte-addresses 0, 4, 8,are taken as the addresses of successive words in the memory.



(a) Big-endian assignment



(b) Little-endian assignment

Consider a 32-bit integer (in hex): 0x12345678 which consists of 4 bytes: 12, 34, 56, and 78.

- Hence this integer will occupy 4 bytes in memory.
- Assume, we store it at memory address starting 1000.
- On little-endian, memory will look like

Address	Value
1000	78
1001	56
1002	34
1003	12

- On big-endian, memory will look like

Address	Value
1000	12
1001	34
1002	56
1003	78

WORD ALIGNMENT

- Words are said to be **Aligned** in memory if they begin at a byte-address that is a multiple of the number of bytes in a word.
- For example,
 - If the word length is 16(2 bytes), aligned words begin at byte-addresses 0, 2, 4
 - If the word length is 64(2 bytes), aligned words begin at byte-addresses 0, 8, 16
- Words are said to have **Unaligned Addresses**, if they begin at an arbitrary byte-address.

ACCESSING NUMBERS, CHARACTERS & CHARACTERS STRINGS

- A number usually occupies one word. It can be accessed in the memory by specifying its word address. Similarly, individual characters can be accessed by their byte-address.
- There are two ways to indicate the length of the string:
 - 1) A special control character with the meaning "end of string" can be used as the last character in the string.
 - 2) A separate memory word location or register can contain a number indicating the length of the string in bytes.

MEMORY OPERATIONS

Two memory operations are:

1. Load (Read/Fetch) &
 2. Store (Write).
- The **Load** operation transfers a copy of the contents of a specific memory-location to the processor. The memory contents remain unchanged.
 - Steps for Load operation:
 - 1) Processor sends the address of the desired location to the memory.
 - 2) Processor issues „read“ signal to memory to fetch the data.
 - 3) Memory reads the data stored at that address.
 - 4) Memory sends the read data to the processor.

- The **Store** operation transfers the information from the register to the specified memory-location. This will destroy the original contents of that memory-location.
- Steps for Store operation are:
 - 1) Processor sends the address of the memory-location where it wants to store data.
 - 2) Processor issues „write“ signal to memory to store the data.
 - 3) Content of register(MDR) is written into the specified memory-location

INSTRUCTIONS & INSTRUCTION SEQUENCING

- A computer must have instructions capable of performing 4 types of operations:
 - 1) Data transfers between the memory and the registers (MOV, PUSH, POP, XCHG).
 - 2) Arithmetic and logic operations on data (ADD, SUB, MUL, DIV, AND, OR, NOT).
 - 3) Program sequencing and control (CALL, RET, LOOP, INT).
 - 4) I/O transfers (IN, OUT).

REGISTER TRANSFER NOTATION (RTN)

The possible locations in which transfer of information occurs are:

- 1) Memory-location 2) Processor register 3) Registers in I/O device.

Location	Hardware Binary Address	Example	Description
Memory	LOC, PLACE, NUM	$R1 \leftarrow [LOC]$	Contents of memory-location LOC are transferred into register R1.
Processor	R0, R1, R2	$[R3] \leftarrow [R1] + [R2]$	Add the contents of register R1 & R2 and places their sum into R3.
I/O Registers	DATAIN, DATAOUT	$R1 \leftarrow DATAIN$	Contents of I/O register DATAIN are transferred into register R1.

ASSEMBLY LANGUAGE NOTATION

To represent machine instructions and programs, assembly language format is used.

Assembly Language Format	Description
Move LOC, R1	Transfer data from memory-location LOC to register R1. The contents of LOC are unchanged by the execution of this instruction, but the old contents of register R1 are overwritten.
Add R1, R2, R3	Add the contents of registers R1 and R2, and places their sum into register R3.

BASIC INSTRUCTION TYPES

Instruction Type	Syntax	Example	Description	Instructions for Operation $C \leftarrow [A] + [B]$
Three Address	Opcode Source1, Source2, Destination	Add A, B, C	Add the contents of memory-locations A & B. Then, place the result into location C.	

Two Address	Opcode Source, Destination	Add A,B	Add the contents of memory-locations A & B. Then, place the result into location B, replacing the original contents of this location. Operand B is both a source and a destination.	Move B, C Add A, C
One Address	Opcode Source/Destination	Load A	Copy contents of memory-location A into accumulator.	Load A Add B Store C
		Add B	Add contents of memory-location B to contents of accumulator register & place sum back into accumulator.	
		Store C	Copy the contents of the accumulator into location C.	
Zero Address	Opcode [no Source/Destination]	Push	Locations of all operands are defined implicitly. The operands are stored in a pushdown stack.	Not possible

Access to data in the registers is much faster than to data stored in memory-locations.

Let R_i represent a general 1-purpose register. The instructions:

Load A, R_i

Store R_i , A

Add A, R_i

are generalizations of the Load, Store and Add Instructions for the single-accumulator case, in which register R_i performs the function of the accumulator.

In processors, where arithmetic operations are allowed only on operands that are in registers, the task $C \leftarrow [A] + [B]$ can be performed by the instruction sequence:

Move A, R_i

Move B, R_j

Add R_i , R_j

Move R_j , C

INSTRUCTION EXECUTION & STRAIGHT LI

- TI

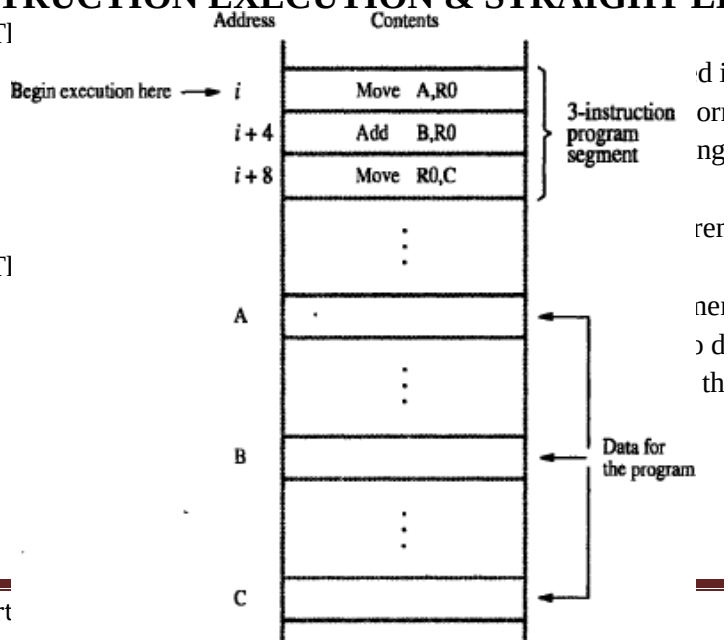


Figure 2.8 A program for $C \leftarrow [A] + [B]$.

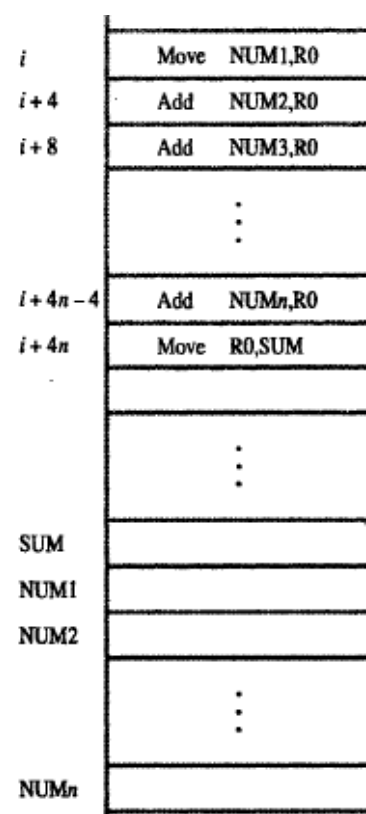


Figure 2.9 A straightline program for adding n numbers.

Program Explanation

- Consider the program for adding a list of n numbers (Figure 2.9).
- The Address of the memory-locations containing the n numbers are symbolically given as NUM1, NUM2.....NUMn.
- Separate Add instruction is used to add each number to the contents of register R0.
- After all the numbers have been added, the result is placed in memory-location SUM.

BRANCHING

- Consider the task of adding a list of „ n “ numbers (Figure 2.10).
- Number of entries in the list „ n “ is stored in memory-location N.
- Register **R1** is used as a counter to determine the number of times the loop is executed.
- Content-location N is loaded into register R1 at the beginning of the program.
- The **Loop** is a straight line sequence of instructions executed as many times as needed. The loop starts at location LOOP and ends at the instruction Branch>0.
- During each pass,
 - address of the next list entry is determined and
 - that entry is fetched and added to R0.
- The instruction *Decrement R1* reduces the contents of R1 by 1 each time through the loop.
- Then **Branch Instruction** loads a new value into the program counter. As a result, the processor fetches and executes the instruction at this new address called the **Branch Target**.
- A **Conditional Branch Instruction** causes a branch only if a specified condition is satisfied. If the condition is not satisfied, the PC is incremented in the normal way, and the next instruction in sequential address order is fetched and executed.

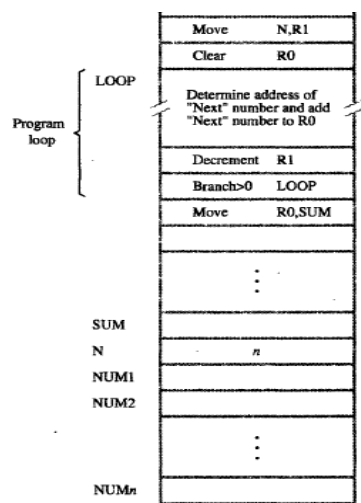


Figure 2.10 Using a loop to add n numbers.

CONDITION CODES

- The processor keeps track of information about the results of various operations. This is accomplished by recording the required information in individual bits, called **Condition Code Flags**.
- These flags are grouped together in a special processor-register called the condition code register (or status register).
- Four commonly used flags are:
 - 1) N (negative) set to 1 if the result is negative, otherwise cleared to 0.
 - 2) Z (zero) set to 1 if the result is 0; otherwise, cleared to 0.
 - 3) V (overflow) set to 1 if arithmetic overflow occurs; otherwise, cleared to 0.
 - 4) C (carry) set to 1 if a carry-out results from the operation; otherwise cleared to 0.

INDIRECTION AND POINTERS

- Instruction does not give the operand or its address explicitly.

- Instead, the instruction provides information from which the new address of the operand can be determined.
- This address is called **Effective Address (EA)** of the operand.

Indirect Mode

- The EA of the operand is the contents of a register(or memory-location).
- The register (or memory-location) that contains the address of an operand is called a **Pointer**.
- We denote the indirection by

→ name of the register or

→ new address given in the instruction.

E.g: *Add (R1),R0* ;The operand is in memory. Register R1 gives the effective-address (B) of the operand. The data is read from location B and added to contents of register R0.

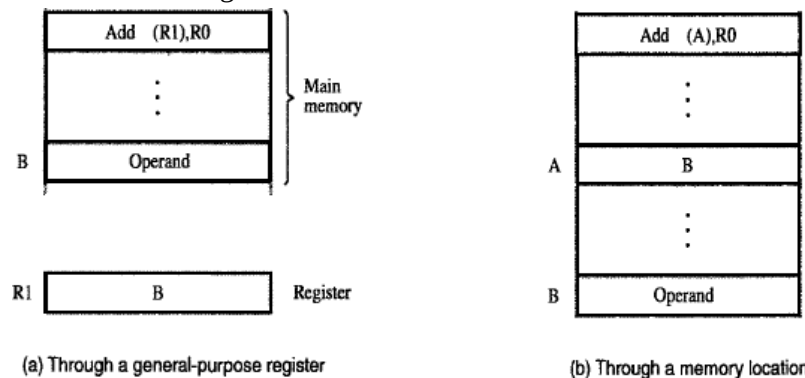


Figure 2.11 Indirect addressing.

- To execute the Add instruction in fig 2.11 (a), the processor uses the value which is in register R1, as the EA of the operand.
- It requests a read operation from the memory to read the contents of location B. The value read is the desired operand, which the processor adds to the contents of register R0.
- Indirect addressing through a memory-location is also possible as shown in fig 2.11(b). In this case, the processor first reads the contents of memory-location A, then requests a second read operation using the value B as an address to obtain the operand.

Address	Contents	
	Move N,R1	} Initialization
	Move #NUM1,R2	
	Clear R0	
→ LOOP	Add (R2),R0	
	Add #4,R2	
	Decrement R1	
	Branch>0 LOOP	
	Move R0,SUM	

Figure 2.12 Use of indirect addressing in the program of Figure 2.10.

Program Explanation

- In above program, Register R2 is used as a pointer to the numbers in the list, and the operands are accessed indirectly through R2.
- The initialization-section of the program loads the counter-value n from memory-location N into R1 and uses the immediate addressing-mode to place the address value NUM1, which is the address of the first number in the list, into R2. Then it clears R0 to 0.
- The first two instructions in the loop implement the unspecified instruction block starting at LOOP.
- The first time through the loop, the instruction Add (R2), R0 fetches the operand at location NUM1 and adds it to R0.
- The second Add instruction adds 4 to the contents of the pointer R2, so that it will contain the

address value NUM2 when the above instruction is executed in the second pass through the loop.

ADDRESSING MODES

- The different ways in which the location of an operand is specified in an instruction are referred to as **Addressing Modes** (Table 2.1).

Table 2.1 Generic addressing modes

Name	Assembler syntax	Addressing function
Immediate	#Value	Operand = Value
Register	R _i	EA = R _i
Absolute (Direct)	LOC	EA = LOC
Indirect	(R _i)	EA = [R _i]
	(LOC)	EA = [LOC]
Index	X(R _i)	EA = [R _i] + X
Base with index	(R _i , R _j)	EA = [R _i] + [R _j]
Base with index and offset	X(R _i , R _j)	EA = [R _i] + [R _j] + X
Relative	X(PC)	EA = [PC] + X
Autoincrement	(R _i)+	EA = [R _i]; Increment R _i
Autodecrement	-(R _i)	Decrement R _i ; EA = [R _i]

EA = effective address
Value = a signed number

IMPLEMENTATION OF VARIABLE AND CONSTANTS

- Variable** is represented by allocating a memory-location to hold its value.
- Thus, the value can be changed as needed using appropriate instructions.
- There are 2 accessing modes to access the variables:
 - 1) Register Mode
 - 2) Absolute Mode

Register Mode

- The operand is the contents of a register.
- The name (or address) of the register is given in the instruction.
- Registers are used as temporary storage locations where the data in a register are accessed.
- For example, the instruction

Move R1, R2 ;Copy content of register R1 into register R2.

Absolute (Direct) Mode

- The operand is in a memory-location.
- The address of memory-location is given explicitly in the instruction.
- The absolute mode can represent global variables in the program.
- For example, the instruction

Move LOC, R2 ;Copy content of memory-location LOC into register R2.

Immediate Mode

- The operand is given explicitly in the instruction.
 - For example, the instruction
- Move #200, R0* ;Place the value 200 in register R0.
- Clearly, the immediate mode is only used to specify the value of a source-operand.

INDIRECTION AND POINTERS

- Instruction does not give the operand or its address explicitly.
- Instead, the instruction provides information from which the new address of the operand can be determined.
- This address is called **Effective Address (EA)** of the operand.

Indirect Mode

- The EA of the operand is the contents of a register(or memory-location).

- The register (or memory-location) that contains the address of an operand is called a **Pointer**.
- We denote the indirection by
 - name of the register or
 - new address given in the instruction.

E.g: *Add (R1),R0* ;The operand is in memory. Register R1 gives the effective-address (B) of the operand. The data is read from location B and added to contents of register R0.

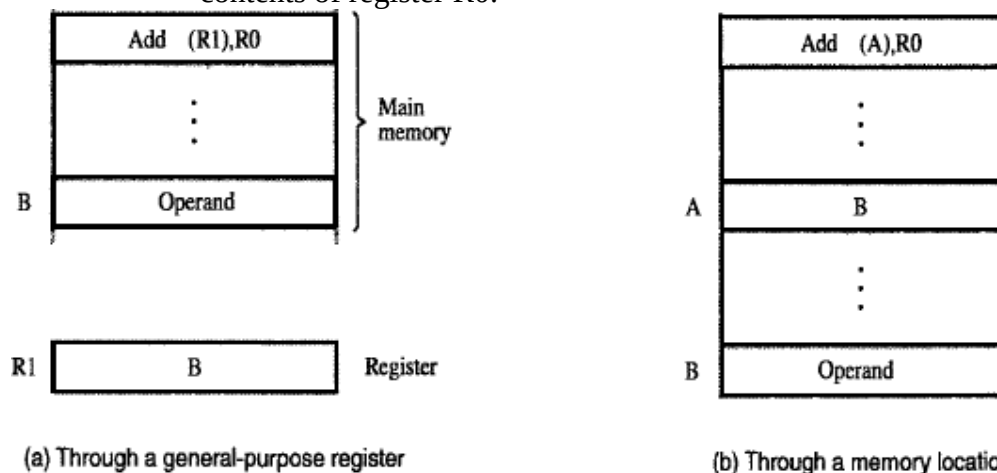


Figure 2.11 Indirect addressing.

- To execute the Add instruction in fig 2.11 (a), the processor uses the value which is in register R1, as the EA of the operand.
- It requests a read operation from the memory to read the contents of location B. The value read is the desired operand, which the processor adds to the contents of register R0.
- Indirect addressing through a memory-location is also possible as shown in fig 2.11(b). In this case, the processor first reads the contents of memory-location A, then requests a second read operation using the value B as an address to obtain the operand.

Address	Contents	
	Move N,R1	} Initialization
	Move #NUM1,R2	
	Clear R0	
→ LOOP	Add (R2),R0	
	Add #4,R2	
	Decrement R1	
	Branch>0 LOOP	
	Move R0,SUM	

Figure 2.12 Use of indirect addressing in the program of Figure 2.10.

Program Explanation

- In above program, Register R2 is used as a pointer to the numbers in the list, and the operands are accessed indirectly through R2.
- The initialization-section of the program loads the counter-value n from memory-location N into R1 and uses the immediate addressing-mode to place the address value NUM1, which is the address of the first number in the list, into R2. Then it clears R0 to 0.
- The first two instructions in the loop implement the unspecified instruction block starting at LOOP.
- The first time through the loop, the instruction Add (R2), R0 fetches the operand at location NUM1 and adds it to R0.
- The second Add instruction adds 4 to the contents of the pointer R2, so that it will contain the address value NUM2 when the above instruction is executed in the second pass through the loop.

INDEXING AND ARRAYS

- A different kind of flexibility for accessing operands is useful in dealing with lists and arrays.

Index mode

- The operation is indicated as $X(R_i)$
 where X =the constant value which defines an offset(also called a displacement).
 R_i =the name of the index register which contains address of a new location.
- The effective-address of the operand is given by $EA = X + [R_i]$
- The contents of the index-register are not changed in the process of generating the effective-address.
- The constant X may be given either
 - as an explicit number or
 - as a symbolic-name representing a numerical value.

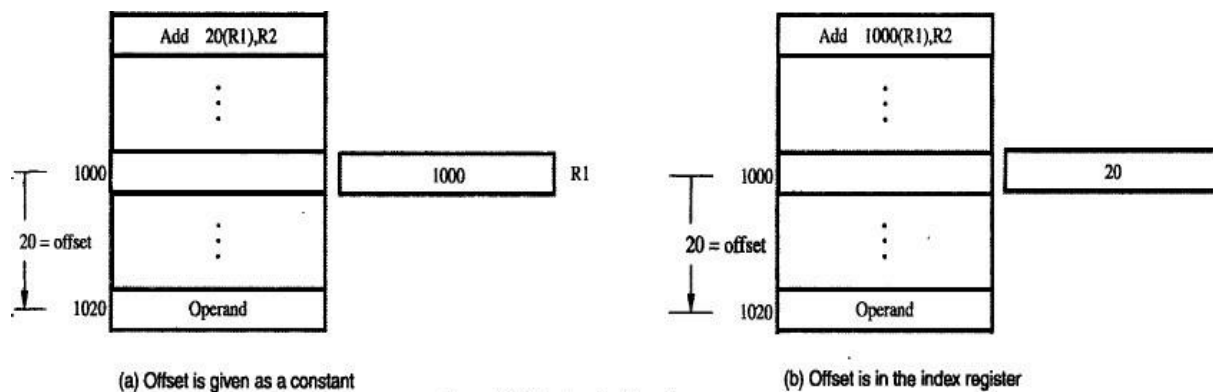


Figure 2.13 Indexed addressing.

- Fig(a) illustrates two ways of using the Index mode. In fig(a), the index register, $R1$, contains the address of a memory-location, and the value X defines an offset(also called a displacement) from this address to the location where the operand is found.
- To find EA of operand: Eg: `Add 20(R1), R2`
 $EA \Rightarrow 1000 + 20 = 1020$
- An alternative use is illustrated in fig(b). Here, the constant X corresponds to a memory address, and the contents of the index register define the offset to the operand. In either case, the effective-address is the sum of two values; one is given explicitly in the instruction, and the other is stored in a register.

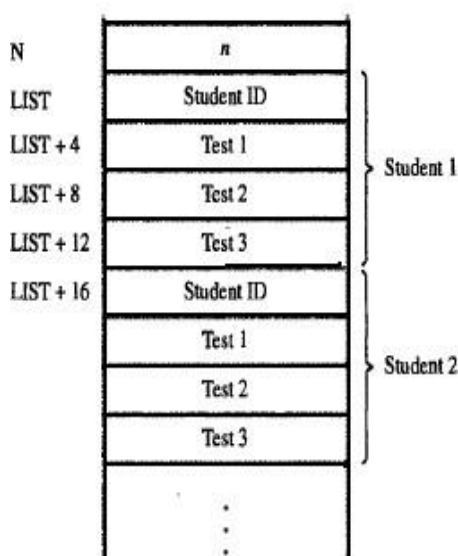


Figure 2.14 A list of students' marks.

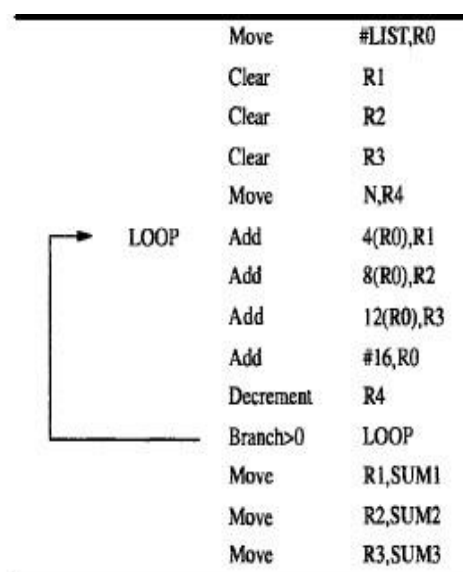


Figure 2.15 Indexed addressing used in accessing test scores in the list in Figure 2.14.

Base with Index Mode

- Another version of the Index mode uses 2 registers which can be denoted as (Ri, Rj)
- Here, a second register may be used to contain the offset X.
- The second register is usually called the *base register*.
- The effective-address of the operand is given by $EA=[Ri]+[Rj]$
- This form of indexed addressing provides more flexibility in accessing operands because both components of the effective-address can be changed.

Base with Index & Offset Mode

- Another version of the Index mode uses 2 registers plus a constant, which can be denoted as X(Ri, Rj)
- The effective-address of the operand is given by $EA=X+[Ri]+[Rj]$
- This added flexibility is useful in accessing multiple components inside each item in a record, where the beginning of an item is specified by the (Ri, Rj) part of the addressing-mode. In other words, this mode implements a 3-dimensional array.

RELATIVE MODE

- This is similar to index-mode with one difference:
- The effective-address is determined using the PC in place of the general purpose register Ri.
- The operation is indicated as X(PC).
- X(PC) denotes an effective-address of the operand which is X locations above or below the current contents of PC.
- Since the addressed-location is identified "relative" to the PC, the name Relative mode is associated with this type of addressing.
- This mode is used commonly in conditional branch instructions.
- An instruction such as
Branch > 0 LOOP ;Causes program execution to go to the branch target location identified by name LOOP if branch condition is satisfied.

ADDITIONAL ADDRESSING MODES

1) Auto Increment Mode

- Effective-address of operand is contents of a register specified in the instruction (Fig: 2.16).
- After accessing the operand, the contents of this register are automatically incremented to point to the next item in a list.
- Implicitly, the increment amount is 1.
- This mode is denoted as $(Ri)+$;where Ri=pointer-register.

2) Auto Decrement Mode

- The contents of a register specified in the instruction are first automatically decremented and are then used as the effective-address of the operand.
- This mode is denoted as $-(Ri)$;where Ri=pointer-register.

- These 2 modes can be used together to implement an important data structure called a stack.

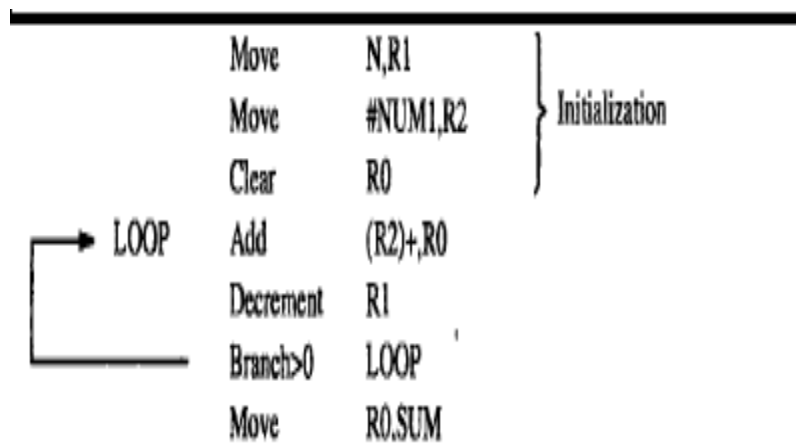


Figure 2.16 The Autoincrement addressing mode used in the program of Figure 2.12.

ASSEMBLY LANGUAGE

- We generally use symbolic-names to write a program.
- A complete set of symbolic-names and rules for their use constitute an **Assembly Language**.
- The set of rules for using the mnemonics in the specification of complete instructions and programs is called the **Syntax** of the language.
- Programs written in an assembly language can be automatically translated into a sequence of machine instructions by a program called an **Assembler**.
- The user program in its original alphanumeric text format is called a **Source Program**, and the assembled machine language program is called an **Object Program**.

For example:

MOVE R0,SUM ;The term MOVE represents OP code for operation performed by instruction.

ADD #5,R3 ;Adds number 5 to contents of register R3 & puts the result back into register R3.

ASSEMBLER DIRECTIVES

- **Directives** are the assembler commands to the assembler concerning the program being assembled.
- These commands are not translated into machine opcode in the object-program.

	Memory address label	Operation	Addressing or data information
Assembler directives	SUM	EQU	200
		ORIGIN	204
	N	DATAWORD	100
	NUM1	RESERVE	400
		ORIGIN	100
Statements that generate machine instructions	START	MOVE	N,R1
		MOVE	#NUM1,R2
		CLR	R0
	LOOP	ADD	(R2),R0
		ADD	#4,R2
		DEC	R1
		BGTZ	LOOP
Assembler directives		MOVE	R0,SUM
		RETURN	
		END	START

Figure 2.18 Assembly language representation for the program in Figure 2.17.